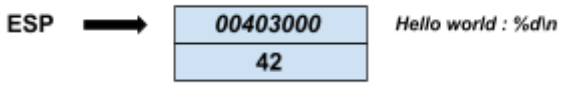
Exercice A - Hello World

Hello World explication du fonctionnement de la pile :

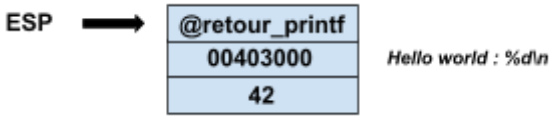
Au début du programme, la pile est vide.

Quand l'instruction ***push 2A*** est exécutée, la valeur 42 est placée sur la pile. ESP est un registre qui pointe toujours le sommet de la pile. Il pointe donc sur la valeur 2A à cet instant.

Ensuite, ***push 00403000*** ajoute l'adresse de la chaîne "Hello World : %d\n" au sommet de la pile. On a donc maintenant, du haut vers le bas : l'adresse de la chaîne puis la valeur 42.

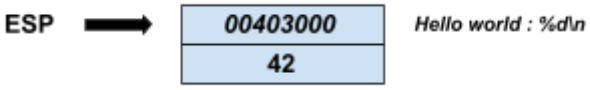


Quand ***call printf*** est exécuté, le processeur empile automatiquement l'adresse de retour. La fonction ***printf*** peut alors lire ses arguments dans la pile : d'abord la chaîne de format, puis la valeur 42.

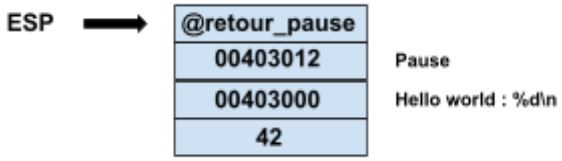


Elle affiche donc "Hello World : 42".

Lorsque ***printf*** se termine, la fonction enlève uniquement l'adresse de retour. Les arguments restent dans la pile. Cette dernière contient donc encore la chaîne et la valeur 42.

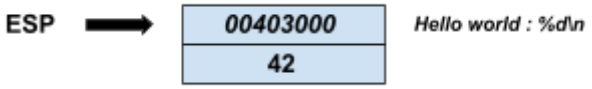


Ensuite, ***push 00403012*** ajoute la chaîne "Pause" sur la pile, puis ***call system*** empile à nouveau une adresse de retour. La fonction ***system*** utilise cet argument pour exécuter la commande ***Pause***. La pile vaut alors :

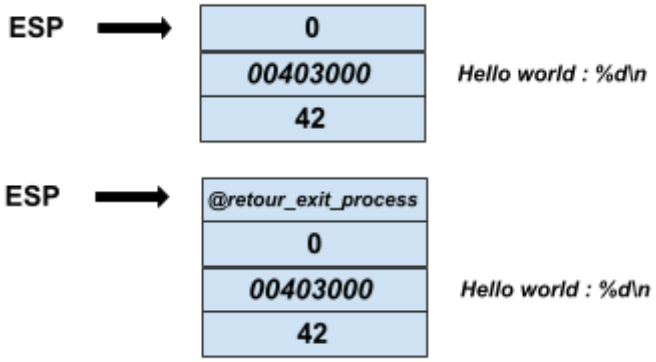


A la fin de la fonction ***system***, on exécute ***add esp, 4***, ce qui dépile "Pause" (car un argument = 4 octets). La fonction ***system*** utilise la convention d'appel **cdecl**, qui demande à ce que le programme appelant nettoie la pile à la fin de l'appel. Cela veut donc dire que lorsque l'appel à la fonction ***system*** est terminé, notre programme doit nettoyer la pile. On

retire donc l'élément "Pause" pour que le programme puisse continuer. Les arguments de ***printf*** sont toujours présents.



Enfin, ***push eax,*** qui a une valeur de 0, place le code de retour sur la pile, puis ***call ExitProcess*** termine le programme. Le 0 est le code de retour de ExitProcess, pour dire que le programme s'est bien déroulé. Cette valeur 0 est donc passée comme argument à ***ExitProcess***. Cela équivaut à ***ExitProcess(0)***. La fonction ***ExitProcess*** ne retourne pas, la pile n'est donc pas nettoyée.



Exercice C.b

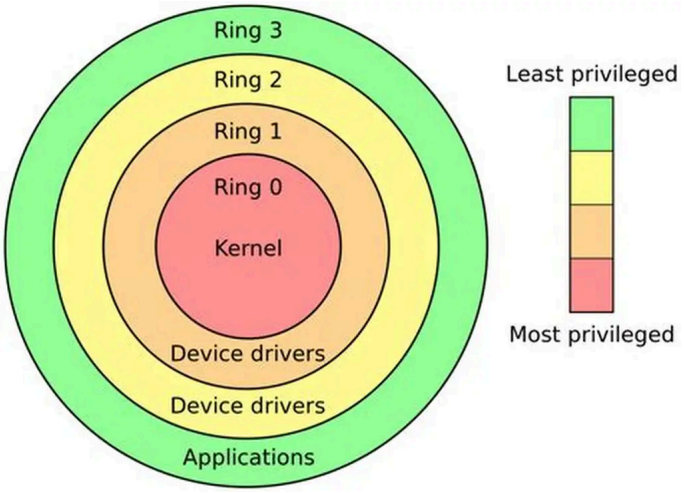
Cette fonction calcule un terme d'une suite bien connue appelée la suite de Fibonacci.

Résumé de l'article

https://www.segmentationfault.fr/reversing/plongeon-dans-les-appels-systemes-windows/

Cet article explique comment fonctionne un appel système sous Windows. Il montre comment un programme qui s'exécute en mode utilisateur (ring 3) peut appeler une fonction du mode noyau (ring 0) puis suit l'exécution d'un programme simple étape par étape jusqu'au noyau du système.

Les niveaux de privilèges des processeurs x86
Les processeurs x86 possèdent plusieurs niveaux de privilèges, appelés rings ou anneaux. Ces niveaux sont codés sur 2 bits, ce qui permet d'avoir 4 niveaux (0 à 3).
Plus le numéro est petit, plus les privilèges sont élevés.



Le ring 0 correspond au mode superviseur (mode noyau), utilisé par les systèmes d'exploitation pour exécuter les instructions critiques. À l'inverse, le ring 3 correspond au mode utilisateur, dans lequel s'exécutent les applications classiques.

Le niveau de privilège actuel du processeur est appelé CPL (Current Privilege Level) et est stocké dans les deux premiers bits des registres CS et SS. Une règle importante est qu'un programme ne peut pas exécuter directement une instruction nécessitant un niveau de privilège plus élevé, ni modifier lui-même ce niveau. Cela pose donc la question : comment un programme utilisateur peut-il appeler une fonction du noyau ?

Exemple de programme et début du traçage

Pour y répondre, il crée un programme simple en C qui appelle la fonction Windows FindFirstFile, permettant d'obtenir le premier fichier d'un dossier. À l'aide d'un débogueur, il observe l'exécution du programme et suit l'appel de cette fonction. L'appel passe d'abord par une zone trampoline (sert de point de saut entre l'appel initial et l'implémentation réelle), qui fait référence à l'IAT (Import Address Table) de l'exécutable. Cette table contient les adresses des fonctions importées. C'est comparable sous Linux aux sections .plt (trampoline)et .got(IAT).

L'exécution arrive ensuite dans la bibliothèque kernel32.dll, où la fonction FindFirstFileA appelle en réalité FindFirstFileExW. Cela s'explique par le fait que les fonctions internes de Windows utilisent Unicode : les fonctions terminées par A gèrent l'ASCII et celles terminées par W l'Unicode.

La fonction finit par appeler des fonctions situées dans ntdll.dll, comme ZwOpenFile. Cette fonction est très courte : elle place dans le registre EAX un numéro d'appel système, puis appelle KiFastSystemCall.

Cette fonction utilise l'instruction SYSENTER, qui permet de passer du mode utilisateur au mode noyau. Historiquement, les appels système étaient réalisés avec des interruptions logicielles (INT 2E sous Windows et INT 80 sous Linux), mais SYSENTER est plus rapide et est devenu la méthode privilégiée.

Fonctionnement de SYSENTER

Contrairement aux interruptions, SYSENTER utilise des registres spéciaux du processeur appelés MSR (Model Specific Registers). Trois registres sont utilisés :

- IA32_SYSENTER_CS : valeur du registre CS à charger lors de l'entrée en mode noyau
- IA32_SYSENTER_ESP : valeur du pointeur de pile
- IA32_SYSENTER_EIP : adresse de la fonction à exécuter en mode noyau

Ces registres permettent de définir directement où et comment le processeur doit basculer en mode noyau.

Traitement de l'appel dans le noyau

Une fois en mode noyau, l'exécution arrive dans la fonction KiFastCallEntry, située dans le noyau Windows (ntoskrnl.exe). Cette fonction effectue des vérifications puis utilise une table appelée SSDT (System Service Dispatch Table).

La SSDT contient les adresses des fonctions du noyau correspondant aux appels systèmes. Le numéro placé dans EAX est utilisé comme index dans cette table. Le système récupère alors l'adresse de la fonction correspondante et l'exécute. Dans l'exemple étudié, le numéro 0x74 correspond à la fonction NtOpenFile, qui est finalement exécutée dans le noyau.

Retour en mode utilisateur

Une fois la fonction noyau exécutée, le système doit revenir en mode utilisateur. Cela se fait via les fonctions KiSystemCallExit ou KiSystemCallExit2, qui utilisent les instructions IRET ou SYSEXIT pour restaurer le contexte d'exécution et revenir au programme utilisateur. Le programme reprend son exécution dans ntdll.dll, puis retourne à l'application initiale. L'article montre le chemin complet d'un appel système sous Windows, depuis un programme en mode utilisateur jusqu'à l'exécution d'une fonction du noyau.

Ce fonctionnement peut être exploité pour réaliser différents types de hooks, par exemple en modifiant l'IAT en mode utilisateur, la SSDT en mode noyau, ou encore en interceptant certaines fonctions du noyau.